PostgreSQL Procedures — Beginner Practice

Designed for your current level: basic CREATE PROCEDURE syntax + parameters + simple SQL logic. No OUT parameters, SELECT INTO, subqueries, CTEs, window functions, or transactions.

Fresh Tables — Run First

```
CREATE TABLE employees (
employee_id SERIAL PRIMARY KEY,
employee_name VARCHAR(100),
department VARCHAR(50),
salary NUMERIC(10,2),
status VARCHAR(20) DEFAULT 'Active'
);

CREATE TABLE products (
product_id SERIAL PRIMARY KEY,
product_name VARCHAR(100),
price NUMERIC(10,2),
stock INT
);

CREATE TABLE students (
student_id SERIAL PRIMARY KEY,
student_name VARCHAR(100),
marks INT,
result VARCHAR(20) DEFAULT 'Pending'
);

INSERT INTO employees (employee_name, department, salary, status) VALUES
('Aarav','IT',55000,'Active'),
('Riya','HR',45000,'Active'),
('Kabir','IT',65000,'Active'),
('Sneha','Finance',50000,'Inactive'),
('Vivek','IT',40000,'Active');

INSERT INTO products (product_name, price, stock) VALUES
('Keyboard',1200,10),
('Mouse',700,20),
('Monitor',9000,5),
('Headphones',2500,0);

INSERT INTO students (student_name, marks, result) VALUES
('Amit',85,'Pending'),
('Neha',42,'Pending'),
('Rahul',67,'Pending'),
('Priya',91,'Pending'),
('Karan',35,'Pending');
```

EASY 1

Update Employee Salary

Create a procedure named **update_salary**. Parameters: employee_id, new_salary. Task: • Update the salary of the given employee. • Test it with at least 3 CALL statements. • After each CALL, use SELECT to verify the change. Practice focus: procedure syntax, IN parameters, UPDATE, CALL.

EASY 2

Change Employee Status

Create a procedure named **change_status**. Parameters: employee_id, new_status. Task: • Update the employee's status. • Allowed values are only **Active** and **Inactive**. • If the supplied status is different, do not update the row. • Use simple IF/ELSE logic. Practice focus: parameters, IF/ELSE, UPDATE, validation.

MEDIUM 3

Add a New Product

Create a procedure named **add_product**. Parameters: product_name, price, stock. Task: • Insert a new product into products. • Price must be greater than 0. • Stock must be 0 or greater. • If values are invalid, do not insert the product. • Test valid and invalid CALL statements. Practice focus: parameters, IF/ELSE, INSERT, validation.

MEDIUM 4

Update Student Result

Create a procedure named **update_result**. Parameter: student_id. Task: • If marks are 40 or more, set result to **Pass**. • If marks are below 40, set result to **Fail**. • Use IF/ELSE. • Update only the result column. • Test students above and below 40. Practice focus: business logic, IF/ELSE, UPDATE.

HARD 5

Product Stock Management

Create a procedure named **manage_stock**. Parameters: product_id, quantity, operation. Operations: • **ADD** → increase stock by quantity. • **REMOVE** → decrease stock by quantity. Rules: • Quantity must be greater than 0. • REMOVE must not make stock negative. • Only ADD and REMOVE are valid operations. • Use IF/ELSIF/ELSE. • Test valid and invalid cases. Practice focus: multiple parameters, branching, UPDATE, validation, combining conditions.

Recommended Learning Order

1. Basic procedure syntax + IN parameters
2. INSERT / UPDATE / DELETE inside procedures
3. IF / ELSIF / ELSE
4. Variables
5. SELECT INTO
6. OUT / INOUT parameters
7. Exception handling

Do Q1–Q5 without looking at solutions. Once these feel comfortable, move to SELECT INTO and OUT parameters.